Отчёт по анализу уязвимостей веб приложения на основе классификации CWE MITRE

Код реализует механизм авторизации пользователя. Имя пользователя и пароль передаются через параметры HTTP запроса. После получения данных пароль хешируется с использованием алгоритма MD5 и выполняется SQL запрос к таблице пользователей базы данных. Если запись найдена, пользователю отображается сообщение об успешном входе и его аватар.

В ходе анализа были выявлены несколько уязвимостей, которые соответствуют различным категориям CWE.

CWE 89 Improper Neutralization of Special Elements used in an SQL Command SQL Injection

Описание уязвимости

SQL запрос формируется путем непосредственной подстановки пользовательских данных в строку запроса. Пользовательские значения username и password добавляются в SQL выражение без фильтрации и без использования параметризованных запросов.

Уязвимый фрагмент логики

Приложение формирует запрос к базе данных используя введенные пользователем значения и передает его напрямую в функцию выполнения запроса.

Причина возникновения

Отсутствует механизм безопасной передачи параметров в SQL запрос. В коде не используются подготовленные выражения и не выполняется экранирование специальных символов.

Потенциальные последствия

Злоумышленник может изменить структуру SQL запроса и выполнить произвольные операции с базой данных. Это может привести к обходу аутентификации, чтению конфиденциальных данных, изменению записей или удалению информации.

Рекомендации по устранению

Для устранения уязвимости необходимо использовать подготовленные выражения и параметризованные запросы. Также следует ограничивать допустимые значения пользовательского ввода.

CWE 327 Use of a Broken or Risky Cryptographic Algorithm

Описание уязвимости

В коде используется алгоритм MD5 для хеширования пароля пользователя. Данный алгоритм считается криптографически устаревшим и небезопасным для хранения паролей.

Причина возникновения

Алгоритм MD5 подвержен атакам подбора и использованию таблиц предварительно вычисленных хешей. Это значительно снижает уровень защиты учетных данных пользователей.

Потенциальные последствия

При получении доступа к базе данных злоумышленник может сравнительно быстро восстановить исходные пароли пользователей.

Рекомендации по устранению

Для хранения паролей необходимо использовать современные алгоритмы хеширования, предназначенные специально для защиты паролей. В языке PHP рекомендуется использовать функции password hash и password verify.

CWE 598 Use of GET Request Method With Sensitive Query Strings

Описание уязвимости

Передача имени пользователя и пароля осуществляется через параметры HTTP метода GET. В результате конфиденциальные данные попадают в строку URL.

Причина возникновения

Метод GET используется для передачи чувствительных данных, которые не должны отображаться в адресной строке браузера.

Потенциальные последствия

Пароль может быть сохранен в истории браузера, логах веб сервера, прокси серверах или системах мониторинга. Это увеличивает риск компрометации учетных данных.

Рекомендации по устранению

Для передачи конфиденциальной информации необходимо использовать метод POST. Также рекомендуется применять защищенное соединение HTTPS.

CWE 79 Improper Neutralization of Input During Web Page Generation Cross Site Scripting

Описание уязвимости

Пользовательские данные выводятся на веб страницу без экранирования. Значение имени пользователя и путь к аватару вставляются в HTML код напрямую.

Причина возникновения

Отсутствует фильтрация или кодирование данных перед их выводом в браузер.

Потенциальные последствия

Злоумышленник может внедрить вредоносный JavaScript код. Это может привести к выполнению сценариев в браузере пользователя, краже сессионных данных или перенаправлению на вредоносные сайты.

Рекомендации по устранению

Перед выводом данных на страницу необходимо применять функции экранирования, например htmlspecialchars. Также рекомендуется проверять корректность данных, полученных из базы данных.

Заключение

Анализ кода показал наличие нескольких распространенных уязвимостей безопасности веб приложений. Основные проблемы связаны с небезопасной обработкой пользовательского ввода, использованием устаревших криптографических алгоритмов и отсутствием защиты при выводе данных на страницу. Для повышения уровня безопасности необходимо использовать подготовленные SQL запросы, современные методы хеширования паролей, передачу данных через метод POST и корректное экранирование данных перед отображением в браузере.